

# Admission Software - Code Explanation (Hinglish)

Yeh folder ek bohot hi detailed OOPs (Object-Oriented Programming) based admission system design karta hai. Yeh University/College me multiple programs, departments, students, normalizers, aur merit generation manage karta hai.

**Sir ko impress karne ke liye main point:** Sir ko batana ki yeh ek dum **"Enterprise/Production level"** ka code hai. Isme humne **"Strategy Design Pattern"** (Normalizers aur Merit Calculators) ka use kiya hai taaki kal ko agar admission ke rules (jaise grading system) change ho toh purana code chedne ki zarurat na pade, bas nayi class bana lo. Aur saara memory management **shared\_ptr** se kiya hai jisse safety milti hai.

## 1. Curriculum.h

Yeh class program/degree ka curriculum define karti hai. (Course me kitne semesters hain aur kya padhaya jayega).

```
#pragma once
#include <string>
#include <vector>

using namespace std;

class Curriculum {
    int totalSemesters; // Kitne sem honge
    int totalCredits;    // Total kitne credits ki degree hai
    vector<string> subjects; // Subjects ki list

public:
    // Constructor initialize karta hai sari values ko
    Curriculum(int totalSemesters, int totalCredits, const vector<string>&
subjects)
        : totalSemesters(totalSemesters), totalCredits(totalCredits),
subjects(subjects) {}
};
```

## 2. Qualification.h

Yeh bohot important hai. Ek student ki pass ki hui ek degree ki details store karta hai (jaise 12th marks, ya GATE score).

```
#pragma once
#include <string>

using namespace std;

class Qualification {
    string degreeName; // Jaise "High School", "BCA"
```

```

double cgpaOrPercentage; // Kitne marks aaye
string institution; // Kahan se ki
int yearOfPassing;
string type; // Type jaise "HIGH_SCHOOL", "BACHELORS", "GATE"
bool specialExamQualified; // Jaise GATE/JEE nikal gaya kya?
double specialExamScore; // Agar GATE diya toh score kya tha?

public:
    // Constructor (specialExam defaults ko false aur 0.0 pe rakhta hai
    agar pass nahi kiye)
    Qualification(string degreeName, double cgpaOrPercentage, string
    institution, int yearOfPassing, string type, bool specialExamQualified =
    false, double specialExamScore = 0.0)
        : degreeName(degreeName), cgpaOrPercentage(cgpaOrPercentage),
    institution(institution), yearOfPassing(yearOfPassing), type(type),
    specialExamQualified(specialExamQualified),
    specialExamScore(specialExamScore) {}

    // Getters
    double getCgpaOrPercentage() const { return cgpaOrPercentage; }
    bool isSpecialExamQualified() const { return specialExamQualified; }
    double getSpecialExamScore() const { return specialExamScore; }
    string getDegreeName() const { return degreeName; }
    string getInstitution() const { return institution; }
    int getYearOfPassing() const { return yearOfPassing; }
    string getType() const { return type; }
};

```

### 3. Student.h

Student ki personal details aur uske qualifications ki ek **list** (vector) store karta hai. (Kyonki ek bache ke paas 10th, 12th, B.Tech teeno ho sakte hain).

```

#pragma once
#include <string>
#include <vector>
#include "Qualification.h"

using namespace std;

class Student {
    string id;
    string name;
    vector<Qualification> qualifications; // Bachhe ki sari degrees ka
    array

public:
    Student(string id, string name) : id(id), name(name) {}

    // Nayi degree/qualification add karne ke liye function
    void addQualification(const Qualification& q) {

```

```

        qualifications.push_back(q);
    }

    // Check karna ki kya bachhe ke paas specific degree (jaise
    "BACHELORS") hai ya nahi
    const Qualification* getQualification(const string& type) const {
        for (const auto& q : qualifications) {
            if (q.getType() == type) {
                return &q; // Pointer return kar diya agar mil gayi
            }
        }
        return nullptr; // Warna Null return karega
    }

    string getId() const { return id; }
    string getName() const { return name; }
};

```

## 4. ScoreNormalizer.h (The Interface)

Alag-alag universities alag grading karti hain. Yeh interface marks ko normalize (ek level par laane) ke liye hai.

```

#pragma once
class ScoreNormalizer {
public:
    // Abstract function: Jo child hoga wo ise override karega
    virtual double normalize(double score) const = 0;
    virtual ~ScoreNormalizer() = default;
};

```

## 5. Normalizer Implementations

Yeh `ScoreNormalizer` ko use karke alag alag logic lagate hain. (Strategy Pattern ka use yahan hai).

```

// PassThroughNormalizer.h
#pragma once
#include "ScoreNormalizer.h"

class PassThroughNormalizer : public ScoreNormalizer {
public:
    double normalize(double score) const override {
        return score; // Jo marks the wahi de diye (No change)
    }
};

// IndianCGPANormalizer.h
#pragma once
#include "ScoreNormalizer.h"

```

```

class IndianCGPANormalizer : public ScoreNormalizer {
public:
    double normalize(double score) const override {
        // Agar CGPA hai (10 se chota), toh usko 9.5 se multiply karke
        Percentage bana do (CBSE/Indian Rule)
        if (score <= 10.0) {
            return score * 9.5;
        }
        return score;
    }
};

```

## 6. Eligibility Checkers

Check karte hain ki admission form accept hoga ya reject.

```

// EligibilityChecker.h (Abstract Interface)
#pragma once
#include <string>
#include "Student.h"

using namespace std;

class EligibilityChecker {
public:
    virtual bool isEligible(const Student& student) const = 0; // Eligible
    hai?
    virtual string getRejectionReason(const Student& student) const = 0; //
    Agar nahi, toh kyu?
    virtual ~EligibilityChecker() = default;
};

// BasicEligibilityChecker.h
#pragma once
#include <string>
#include <memory>
#include "EligibilityChecker.h"
#include "ScoreNormalizer.h"

using namespace std;

class BasicEligibilityChecker : public EligibilityChecker {
    string requiredType; // Kaunsi degree chahiye? (E.g. "HIGH_SCHOOL")
    double minPercentage; // Minimum kitne %? (E.g. 50%)
    shared_ptr<ScoreNormalizer> normalizer; // Marks theek karne wali
    machine

public:
    BasicEligibilityChecker(string requiredType, double minPercentage,
        shared_ptr<ScoreNormalizer> normalizer)
        : requiredType(requiredType), minPercentage(minPercentage),

```

```

normalizer(normalizer) {}

// Check eligibility
bool isEligible(const Student& student) const override {
    const Qualification* q = student.getQualification(requiredType);
    if (!q) return false; // Degree hi nahi hai toh seedha reject

    // Marks theek karke check karo ki minimum % cross kar raha hai kya
    double score = normalizer->normalize(q->getCgpaOrPercentage());
    return score >= minPercentage;
}

// Reason dena (Kyu reject hua)
string getRejectionReason(const Student& student) const override {
    const Qualification* q = student.getQualification(requiredType);
    if (!q) return "Missing required qualification";

    double score = normalizer->normalize(q->getCgpaOrPercentage());
    if (score < minPercentage) {
        return "Minimum percentage not met. Required: " +
to_string(minPercentage) + "%, Found: " + to_string(score) + "%";
    }
    return "Eligible";
}
};

```

## 7. Merit Calculators

Rank list banane ke liye scores assign karte hain.

```

// MeritCalculator.h (Abstract Interface)
#pragma once
#include "Student.h"

class MeritCalculator {
public:
    virtual double calculateMerit(const Student& student) const = 0;
    virtual ~MeritCalculator() = default;
};

// StandardMeritCalculator.h (Normal ranking)
#pragma once
#include <string>
#include <memory>
#include "MeritCalculator.h"
#include "ScoreNormalizer.h"

using namespace std;

class StandardMeritCalculator : public MeritCalculator {
    string baseQualification;

```

```

        shared_ptr<ScoreNormalizer> normalizer;

public:
    StandardMeritCalculator(string baseQualification,
        shared_ptr<ScoreNormalizer> normalizer)
        : baseQualification(baseQualification), normalizer(normalizer) {}

    double calculateMerit(const Student& student) const override {
        const Qualification* q =
student.getQualification(baseQualification);
        if (!q) return 0.0;

        // Seedha normalized percentage pass kardo as a Rank Score
        return normalizer->normalize(q->getCgpaOrPercentage());
    }
};

// GatePreferenceMeritCalculator.h (M.Tech rank logic)
#pragma once
#include <memory>
#include "MeritCalculator.h"
#include "ScoreNormalizer.h"

using namespace std;

class GatePreferenceMeritCalculator : public MeritCalculator {
    shared_ptr<ScoreNormalizer> normalizer;

public:
    GatePreferenceMeritCalculator(shared_ptr<ScoreNormalizer> normalizer) :
normalizer(normalizer) {}

    // Agar bachhe ne GATE kiya hai toh score badh jayega.
    double calculateMerit(const Student& student) const override {
        const Qualification* bachelors =
student.getQualification("BACHELORS");
        if (!bachelors) return 0.0;

        double baseScore = normalizer->normalize(bachelors-
>getCgpaOrPercentage());

        const Qualification* gate = student.getQualification("GATE");
        // Rule: UG ka 60% weightage, GATE ka 40% weightage mila ke final
score banega.
        if (gate && gate->isSpecialExamQualified()) {
            return (baseScore * 0.6) + (gate->getSpecialExamScore() * 0.4);
        }
        // Agar GATE clear nahi kiya toh bas UG ka 60% milega (Merit gir
jayegi)
        return baseScore * 0.6;
    }
};

```

## 8. Program & Department Classes

College ka structure banate hain.

```
// Program.h
#pragma once
#include <string>
#include <memory>
#include "Curriculum.h"
#include "EligibilityChecker.h"

using namespace std;

class Program {
    string name; // Jaise BCA
    string type; // Jaise UG
    shared_ptr<Curriculum> curriculum; // Iska syllabus (Composition)
    shared_ptr<EligibilityChecker> eligibilityChecker; // Isme admission ka
    rule (Composition)

public:
    Program(string name, string type, shared_ptr<Curriculum> curriculum,
shared_ptr<EligibilityChecker> eligibilityChecker)
        : name(name), type(type), curriculum(curriculum),
eligibilityChecker(eligibilityChecker) {}

    string getName() const { return name; }
    string getType() const { return type; }
    shared_ptr<Curriculum> getCurriculum() const { return curriculum; }
    shared_ptr<EligibilityChecker> getEligibilityChecker() const { return
eligibilityChecker; }
};

// Department.h
#pragma once
#include <string>
#include <vector>
#include <memory>
#include "Program.h"

using namespace std;

class Department {
    string name; // Jaise "Computer Science"
    vector<shared_ptr<Program>> offeredPrograms; // Program ki list jo ye
department chalata hai

public:
    Department(string name) : name(name) {}

    void addProgram(shared_ptr<Program> program) {
        offeredPrograms.push_back(program);
    }
};
```

```

    }

    string getName() const { return name; }
};

```

## 9. AdmissionProcess.h

Asli process class, yeh forms leti hai, reject karti hai aur merit banati hai.

```

#pragma once
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <algorithm>
#include "Program.h"
#include "Department.h"
#include "MeritCalculator.h"
#include "Student.h"

using namespace std;

class AdmissionProcess {
    int academicYear;
    shared_ptr<Program> program;
    shared_ptr<Department> department;
    int seatCapacity; // Total khali seats
    shared_ptr<MeritCalculator> meritCalculator; // Ranking system
    vector<Student*> applicants;
    vector<Student*> admittedStudents;

public:
    AdmissionProcess(int academicYear, shared_ptr<Program> program,
        shared_ptr<Department> department, int seatCapacity,
        shared_ptr<MeritCalculator> meritCalculator)
        : academicYear(academicYear), program(program),
        department(department), seatCapacity(seatCapacity),
        meritCalculator(meritCalculator) {}

    // Form Apply Process
    void apply(Student& student) {
        auto checker = program->getEligibilityChecker();
        // Check karta hai eligible hai ya nahi
        if (checker && !checker->isEligible(student)) {
            // Nahi hai toh seedha print "Application Rejected" and Reason
            cout << "Application Rejected for " << student.getName() << " -
> " << checker->getRejectionReason(student) << endl;
            return;
        }
        cout << "Application Accepted for " << student.getName() << " to "
<< program->getName() << endl;
    }

```

```

        applicants.push_back(&student);
    }

    // Merit Ranking
    void generateMeritListAndAdmit() {
        // Merit calculator se score mangwa kar Sort function ko call kiya
        sort(applicants.begin(), applicants.end(), [this](Student* s1,
Student* s2) {
            return meritCalculator->calculateMerit(*s1) > meritCalculator-
>calculateMerit(*s2);
        });

        cout << "--- Merit List for " << program->getName() << " (" <<
academicYear << ") ---" << endl;
        int count = 0;
        for (Student* s : applicants) {
            double score = meritCalculator->calculateMerit(*s);
            cout << s->getName() << " - Score: " << score << endl;

            // Seat mili toh admit ho gaya
            if (count < seatCapacity) {
                admittedStudents.push_back(s);
                count++;
            }
        }
        cout << "-----" <<
endl;
    }

    // Sirf admit hue students ke naam print karna
    void printAdmittedStudents() const {
        cout << "Admitted Students to " << program->getName() << " (" <<
academicYear << "):" << endl;
        for (Student* s : admittedStudents) {
            cout << "- " << s->getName() << endl;
        }
    }
};

```

## 10. main.cpp

Yeh driver file hai jahan sab object bante hain.

```

#include <iostream>
#include <memory>
#include <vector>
#include <string>
#include "Qualification.h"
#include "Student.h"
#include "EligibilityChecker.h"
#include "ScoreNormalizer.h"

```

```
#include "IndianCGPANormalizer.h"
#include "PassThroughNormalizer.h"
#include "BasicEligibilityChecker.h"
#include "Curriculum.h"
#include "Program.h"
#include "Department.h"
#include "MeritCalculator.h"
#include "StandardMeritCalculator.h"
#include "GatePreferenceMeritCalculator.h"
#include "AdmissionProcess.h"

using namespace std;

int main() {
    cout << "=== Generalized Student Admission & Program Registration
System ===" << endl;

    // 1. Department Banaya
    auto scsitDept = make_shared<Department>("School of Computer Science &
IT (DAVV)");

    // 2. Syllabus (Curriculum) Banaya
    auto bcaCurriculum = make_shared<Curriculum>(6, 120, vector<string>
{"Programming in C", "Database Management", "Web Technologies"});
    auto mtechCsCurriculum = make_shared<Curriculum>(4, 80, vector<string>
{"Advanced ML", "Distributed Systems", "Cloud Computing"});
    auto mcaCurriculum = make_shared<Curriculum>(4, 80, vector<string>
{"Advanced Java", "Cloud Computing", "Software Engineering"});

    // Normalizer rule set kiya ki CGPA ko 9.5 se multiply karo
    auto indianNormalizer = make_shared<IndianCGPANormalizer>();

    // 3. Rules banaye ki UG (Undergrad) aur PG (Postgrad) ko form bharne
ke liye kya chahiye
    auto ugEligibility = make_shared<BasicEligibilityChecker>
("HIGH_SCHOOL", 50.0, indianNormalizer); // 12th me 50%
    auto pgEligibility = make_shared<BasicEligibilityChecker>("BACHELORS",
60.0, indianNormalizer); // Graduation me 60%

    // 4. Final Programs setup kiye aur Department me dal diye
    auto bca = make_shared<Program>("BCA", "UG", bcaCurriculum,
ugEligibility);
    auto mca = make_shared<Program>("MCA", "PG", mcaCurriculum,
pgEligibility);
    auto mtechCS = make_shared<Program>("M.Tech Computer Science", "PG",
mtechCsCurriculum, pgEligibility);

    scsitDept->addProgram(bca);
    scsitDept->addProgram(mca);
    scsitDept->addProgram(mtechCS);

    // 5. Dummy Candidates (Students) Banaye
    Student s1("S101", "Alice");
    s1.addQualification(Qualification("High School", 92.5, "DPS", 2023,
```

```

"HIGH_SCHOOL")); // Sirf 12th pass hai

Student s2("S102", "Bob");
s2.addQualification(Qualification("High School", 48.0, "KVS", 2023,
"HIGH_SCHOOL")); // Yeh reject hoga kyunki % kam hai (50 se kam)

Student s3("S103", "Charlie");
s3.addQualification(Qualification("High School", 85.0, "State Board",
2020, "HIGH_SCHOOL"));
s3.addQualification(Qualification("BCA", 8.5, "DAVV", 2023,
"BACHELORS")); // PG ke liye eligible hai (Degree bhi hai)

Student s4("S104", "David");
s4.addQualification(Qualification("High School", 90.0, "CBSE", 2019,
"HIGH_SCHOOL"));
s4.addQualification(Qualification("B.Tech", 9.2, "IIT", 2023,
"BACHELORS"));
s4.addQualification(Qualification("GATE", 75.0, "GATE Board", 2023,
"GATE", true, 75.0)); // Isne GATE clear kiya hai

// 6. Admission Process (Har program ki seats aur merit criteria
allocate kiya)
AdmissionProcess bcaAdmission(2024, bca, scsitDept, 1,
make_shared<StandardMeritCalculator>("HIGH_SCHOOL", indianNormalizer));
AdmissionProcess mcaAdmission(2024, mca, scsitDept, 1,
make_shared<StandardMeritCalculator>("BACHELORS", indianNormalizer));
// M.Tech me rank banane ka special formula lagega
(GatePreferenceMeritCalculator)
AdmissionProcess mtechCsAdmission(2024, mtechCS, scsitDept, 2,
make_shared<GatePreferenceMeritCalculator>(indianNormalizer));

cout << "\n--- Applying to Programs ---" << endl;

// 7. Bacho ne apply kiya
bcaAdmission.apply(s1);
bcaAdmission.apply(s2); // Bob yahan reject hoga, error print hoga
mcaAdmission.apply(s3);
mtechCsAdmission.apply(s4); // David ka GATE score help karega

cout << "\n--- Generating Merit Lists & Admissions ---" << endl;

// 8. Result nikalo
bcaAdmission.generateMeritListAndAdmit();
bcaAdmission.printAdmittedStudents();
cout << endl;

mcaAdmission.generateMeritListAndAdmit();
mcaAdmission.printAdmittedStudents();
cout << endl;

mtechCsAdmission.generateMeritListAndAdmit();
mtechCsAdmission.printAdmittedStudents();
cout << endl;

```

```
    return 0;  
}
```